WEB & APP DEVELOPMENT

REST APIs

Design, secure, document, and test resource-oriented HTTP APIs the way production teams and interviewers expect.

rest · http · api-design · json · jwt · oauth · openapi

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. What REST Is and Why It Matters
2. Setup and Getting Started
3. Resources and URI Design
4. HTTP Methods and Semantics
5. Status Codes
6. Request and Response Anatomy
7. Authentication and Authorization
8. Versioning, Pagination, and Idempotency
9. A Worked Mini-Example: A Tasks API
10. Designing and Documenting with OpenAPI
11. Testing REST APIs
12. Best Practices and Common Pitfalls
13. Interview Questions & Answers
14. Further Learning

What REST Is and Why It Matters

REST (Representational State Transfer) is an architectural style for networked applications. It is not a protocol or a standard — it is a set of constraints that, when applied over HTTP, produce APIs that are scalable, cacheable, and simple to reason about. Nearly every public web API you have ever used speaks REST.

Coined by Roy Fielding in his 2000 doctoral dissertation, REST describes how the Web itself is built. The core idea: expose your application's data and functionality as **resources**, give each resource a

stable address (a URI), and manipulate those resources using a small, uniform set of HTTP verbs. The client and server exchange *representations* of resources — most commonly JSON — rather than calling remote procedures directly.

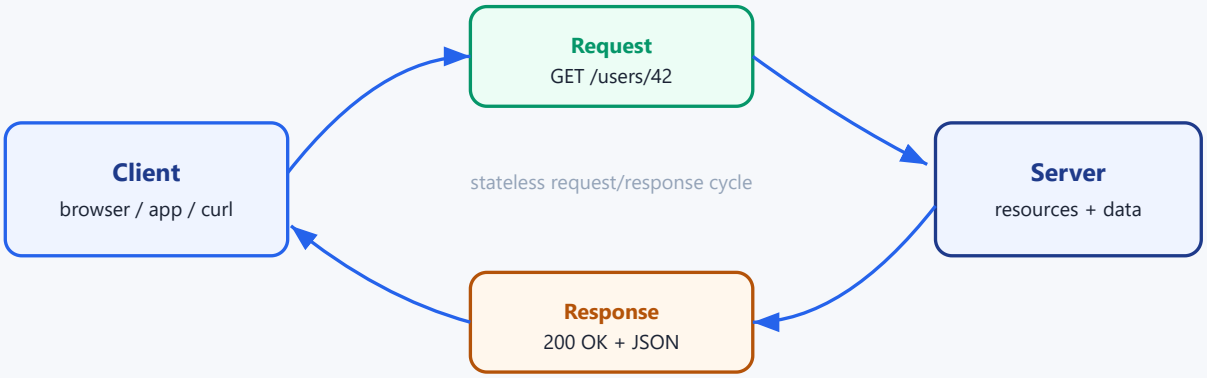


Fig 1. The REST request/response cycle: the client addresses a resource by URI and a verb; the server returns a representation and a status code.

> **Note**
> "RESTful" is a spectrum, not a checkbox. Most real-world APIs follow the pragmatic conventions here (resources, verbs, status codes, JSON) without implementing every one of Fielding's constraints — notably HATEOAS, which is rare in practice.

The Six REST Constraints

CLIENT-SERVER

Separation of concerns: UI lives on the client, data on the server. Each evolves independently.

STATELESS

Every request carries all context needed. The server stores no session state between requests.

CACHEABLE

Responses declare whether they can be cached, improving performance and scalability.

UNIFORM INTERFACE

Resources, standard verbs, and self-descriptive messages — the heart of REST.

LAYERED SYSTEM

Clients cannot tell whether they talk to the origin server or an intermediary (proxy, gateway, CDN).

CODE ON DEMAND

Optional: servers may send executable code (e.g. JavaScript) to extend the client.

Setup and Getting Started

You do not need a framework to *call* a REST API — just an HTTP client. To *build* one, pick a language and web framework. Below are the tools you will reach for constantly.

BASH

```bash
# ——— Calling an API with curl ———
# GET a resource
curl https://api.example.com/v1/users/42

# GET with headers (auth + content negotiation)
curl -H "Authorization: Bearer $TOKEN" \
     -H "Accept: application/json" \
     https://api.example.com/v1/users/42

# POST JSON to create a resource
curl -X POST https://api.example.com/v1/users \
     -H "Content-Type: application/json" \
     -d '{"name":"Ada","email":"ada@example.com"}'

# PATCH to partially update; -i shows response headers + status line
curl -i -X PATCH https://api.example.com/v1/users/42 \
     -H "Content-Type: application/json" \
     -d '{"name":"Ada Lovelace"}'

# DELETE
curl -X DELETE https://api.example.com/v1/users/42 \
     -H "Authorization: Bearer $TOKEN"
```

BASH

```
# ——— Spin up a tiny REST server to experiment ———
# Node.js + Express
npm init -y
npm install express
# Python + FastAPI
pip install fastapi uvicorn
# httpie: a friendlier curl for exploration
pip install httpie
http GET api.example.com/v1/users/42 Authorization:"Bearer $TOKEN"
```

Tip

Keep a REST client like **Postman**, **Insomnia**, or VS Code's **REST Client** extension handy. Save requests as collections so you can replay auth flows and share them with teammates.

Resources and URI Design

A resource is any "thing" your API exposes: a user, an order, a photo, a search result. URIs are the names of those things. Good URI design is the single highest-leverage decision in an API — it is the part clients see most and the part hardest to change later.

Rules of Thumb

NOUNS, NOT VERBS

The verb lives in the HTTP method. Use `/orders`, never `/getOrders` or `/createOrder`.

PLURAL COLLECTIONS

Use `/users` for the collection and `/users/42` for a member. Consistency beats cleverness.

HIERARCHY FOR RELATIONSHIPS

`/users/42/orders` reads as "the orders belonging to user 42".

LOWERCASE, HYPHENATED

`/shipping-addresses`, not `/shippingAddresses` or `/Shipping_Addresses`.

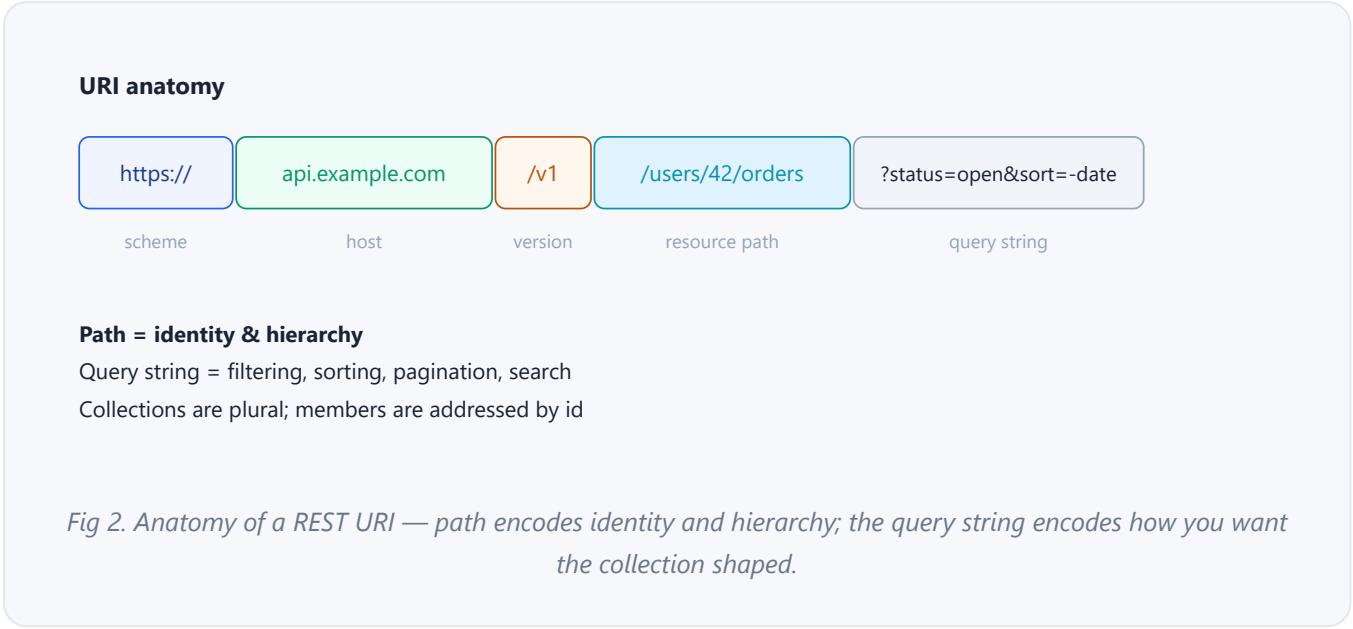


Fig 2. Anatomy of a REST URI — path encodes identity and hierarchy; the query string encodes how you want the collection shaped.

Intent	Method + URI	Meaning
List users	`GET /users`	Collection of all users
Get one user	`GET /users/42`	A single member
Create user	`POST /users`	Server assigns the id
Replace user	`PUT /users/42`	Full representation
Update fields	`PATCH /users/42`	Partial update
Delete user	`DELETE /users/42`	Remove the member
Sub-collection	`GET /users/42/orders`	Related resources
Filter/search	`GET /orders?status=open`	Query parameters

HTTP Methods and Semantics

The uniform interface boils down to a handful of verbs. Two properties define their behaviour: **safe** methods never modify server state, and **idempotent** methods produce the same result whether called once or many times.

Method	Purpose	Safe	Idempotent	Request body
`GET`	Read a resource	yes	yes	no
`HEAD`	Read headers only	yes	yes	no
`POST`	Create / process	no	no	yes
`PUT`	Replace entirely	no	yes	yes
`PATCH`	Partial update	no	not required	yes
`DELETE`	Remove a resource	no	yes	no
`OPTIONS`	Discover capabilities	yes	yes	no

PUT vs PATCH

Both update, but differently. `PUT` sends the *complete* new representation and replaces the resource; omitted fields are cleared. `PATCH` sends only the fields that change.

HTTP

```
PUT /users/42 HTTP/1.1
Content-Type: application/json

{ "name": "Ada", "email": "ada@example.com", "role": "admin" }
# Any field not sent is removed/reset — full replacement.

PATCH /users/42 HTTP/1.1
Content-Type: application/json

{ "role": "admin" }
# Only role changes; name and email are untouched.
```

⚠ **Watch out**

`POST` is not idempotent. Submitting the same "create order" POST twice creates two orders. For retry-safe creation, use an idempotency key (covered later).

Status Codes

The status code is the first thing a client inspects. Use the *right* code — do not return `200 OK` with an error buried in the body. The number carries machine-readable meaning.

2XX SUCCESS

`200` OK · `201` Created · `202` Accepted · `204` No Content

3XX REDIRECT

`301` Moved · `304` Not Modified · `307` / `308` Temporary/Permanent

4XX CLIENT ERROR

`400` · `401` · `403` · `404` · `409` · `422` · `429`

5XX SERVER ERROR

`500` Internal · `502` Bad Gateway · `503` Unavailable · `504` Timeout

Code	Name	When to use it
`200`	OK	Successful GET/PUT/PATCH returning a body
`201`	Created	POST created a resource; return a `Location` header
`204`	No Content	Success with no body (e.g. DELETE)
`400`	Bad Request	Malformed syntax or invalid parameters
`401`	Unauthorized	Missing or invalid authentication
`403`	Forbidden	Authenticated but not permitted
`404`	Not Found	Resource does not exist
`409`	Conflict	State conflict, e.g. duplicate or version mismatch
`422`	Unprocessable Entity	Syntactically valid but semantically invalid
`429`	Too Many Requests	Rate limit exceeded; send `Retry-After`
`500`	Internal Server Error	Unhandled server-side failure
`503`	Service Unavailable	Overloaded or down for maintenance

✦ **401 vs 403**

`401` means "I don't know who you are — authenticate." `403` means "I know who you are, but you may not do this." Mixing them up is a classic bug.

Request and Response Anatomy

An HTTP message has a start line, headers, a blank line, and an optional body. Mastering the common headers is most of the job.

HTTP

```http
POST /v1/orders HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsIn...
Content-Type: application/json
Accept: application/json
Idempotency-Key: 7f3c-9a21-b004

{ "productId": "sku-991", "quantity": 2 }
```

HTTP

```http
HTTP/1.1 201 Created
Content-Type: application/json
Location: /v1/orders/5581
Cache-Control: no-store
X-RateLimit-Remaining: 59

{
  "id": 5581,
  "productId": "sku-991",
  "quantity": 2,
  "status": "pending",
  "createdAt": "2026-10-01T09:12:44Z"
}
```

Header	Direction	Role
`Content-Type`	both	Media type of the body (`application/json`)
`Accept`	request	Media types the client can handle
`Authorization`	request	Credentials (Bearer token, Basic, etc.)
`Location`	response	URI of a newly created resource
`ETag`	response	Version tag for caching / concurrency
`Cache-Control`	both	Caching directives
`Retry-After`	response	How long to wait after 429/503

JSON: The Lingua Franca

JSON is the default representation for REST. It is human-readable, language-agnostic, and maps cleanly onto objects. Keep key names consistent (pick `camelCase` or `snake_case` and never mix), use ISO-8601 for timestamps, and return numbers as numbers, not strings.

JSON

```json
{
  "id": 42,
  "name": "Ada Lovelace",
  "email": "ada@example.com",
  "isActive": true,
  "roles": ["admin", "editor"],
  "createdAt": "2026-10-01T09:12:44Z",
  "address": { "city": "London", "country": "UK" }
}
```

> **Error envelopes**
> Standardize errors. RFC 9457 "Problem Details" (`application/problem+json`) is the modern standard: a body with `type` , `title` , `status` , `detail` , and `instance` .

JSON

```json
{
  "type": "https://api.example.com/errors/validation",
  "title": "Validation failed",
  "status": 422,
  "detail": "quantity must be a positive integer",
  "instance": "/v1/orders",
  "errors": [{ "field": "quantity", "message": "must be ≥ 1" }]
}
```

Authentication and Authorization

Authentication proves *who* you are; authorization decides *what* you may do. Three schemes dominate REST APIs.

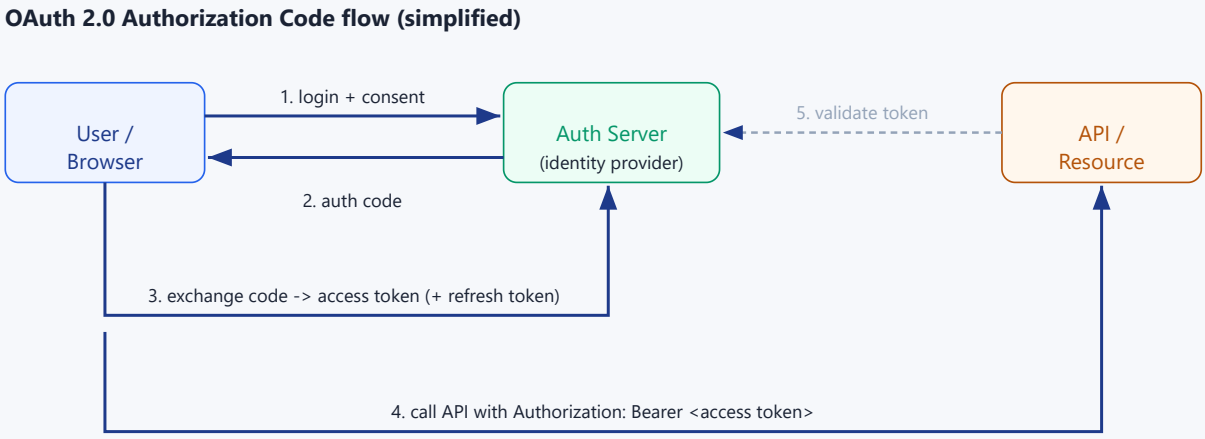


Fig 3. OAuth 2.0 Authorization Code flow: the API never sees the user's password; it trusts a short-lived access token minted by the auth server.

API KEYS

A single opaque string identifying the caller. Simple, good for server-to-server and public data. Weakness: no built-in expiry or scopes; treat like a password.

JWT (BEARER)

A signed, self-contained token carrying claims (user id, scopes, expiry). The server verifies the signature — no DB lookup. Stateless and fast.

OAUTH 2.0

A delegation framework. Lets a third-party app act on a user's behalf without seeing their password. OIDC adds an identity layer on top.

JWT Up Close

A JWT has three Base64URL parts separated by dots: `header.payload.signature`. The signature is computed over the header and payload with a secret (HMAC) or private key (RSA/ECDSA), so tampering is detectable.

JAVASCRIPT

```
// Node.js — issue and verify a JWT with jsonwebtoken
import jwt from "jsonwebtoken";

const SECRET = process.env.JWT_SECRET;

// Sign a token at login (short-lived access token)
const token = jwt.sign(
  { sub: "42", scope: "orders:read orders:write" },
  SECRET,
  { expiresIn: "15m", issuer: "api.example.com" }
);

// Verify on each protected request (Express middleware)
function auth(req, res, next) {
  const header = req.headers.authorization || "";
  const [scheme, value] = header.split(" ");
  if (scheme !== "Bearer" || !value) {
    return res.status(401).json({ title: "Missing bearer token" });
  }
  try {
    req.user = jwt.verify(value, SECRET);   // throws if invalid/expired
    next();
  } catch (err) {
    return res.status(401).json({ title: "Invalid or expired token" });
  }
}
```

Danger

Never put secrets or sensitive PII in a JWT payload — it is only Base64-encoded, not encrypted, and anyone holding the token can read it. Keep access tokens short-lived and rotate with refresh tokens. Always serve APIs over HTTPS.

Scheme	Best for	Expiry	Scopes	Stateless
API Key	Internal / public read	manual	limited	yes
JWT	SPAs, microservices	built-in (`exp`)	via claims	yes
OAuth 2.0	Third-party delegation	built-in	rich	token-dependent

Versioning, Pagination, and Idempotency

Versioning

APIs change. Versioning lets you evolve without breaking existing clients. The three common strategies trade off visibility against purity.

Strategy	Example	Pros / Cons
URI path	`/v1/users`	Obvious, cache-friendly; "impure" per REST
Header	`Accept: application/vnd.ex.v2+json`	Clean URIs; harder to test/debug
Query param	`/users?version=2`	Easy to default; clutters the query string

✦ **Pragmatic choice**

URI path versioning (`/v1`) is the most widely used because it is visible, easy to route, and trivial to curl. Only bump the major version for breaking changes; add fields additively within a version.

Pagination

Never return an unbounded list. Two dominant styles:

OFFSET / LIMIT

`GET /users?limit=20&offset=40`. Simple and jumpable, but slow and inconsistent on large, changing datasets.

CURSOR / KEYSET

`GET /users?limit=20&cursor=eyJpZCI6NDB9`. Stable and fast at scale; cannot jump to an arbitrary page.

JSON

```
{
  "data": [ { "id": 41 }, { "id": 42 } ],
  "pagination": {
    "limit": 20,
    "nextCursor": "eyJpZCI6NjB9",
    "hasMore": true
  }
}
```

Idempotency Keys

To make non-idempotent `POST` requests safe to retry (network glitch, timeout), the client sends a unique `Idempotency-Key`. The server records the key with the first result and replays that stored result for any repeat — so a double-submit never creates two orders.

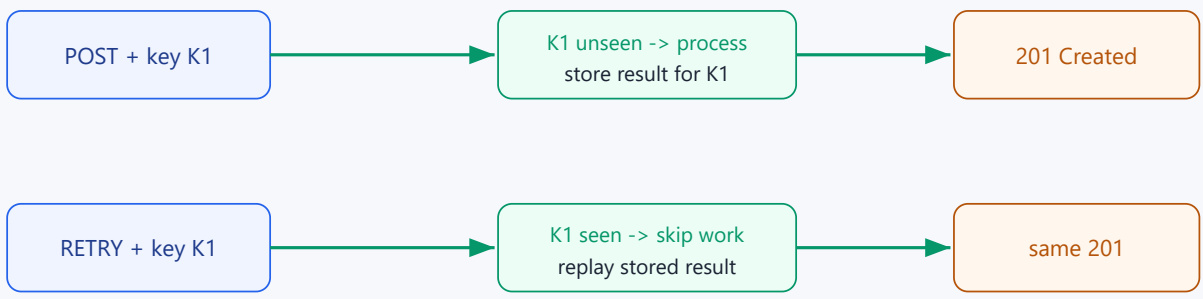


Fig 4. Idempotency keys make POST retry-safe: the second call with the same key replays the first result instead of creating a duplicate.

A Worked Mini-Example: A Tasks API

Let us build a small, idiomatic CRUD endpoint set with FastAPI. It demonstrates resources, status codes, validation, and pagination in under 60 lines.

PYTHON

```python
from fastapi import FastAPI, HTTPException, Response, status
from pydantic import BaseModel, Field
from uuid import uuid4

app = FastAPI(title="Tasks API", version="1.0.0")
db: dict[str, dict] = {}   # in-memory store for the demo

class TaskIn(BaseModel):
    title: str = Field(min_length=1, max_length=120)
    done: bool = False

class Task(TaskIn):
    id: str

@app.get("/v1/tasks")                       # list + simple pagination
def list_tasks(limit: int = 20, offset: int = 0):
    items = list(db.values())
    return {"data": items[offset: offset + limit],
            "pagination": {"limit": limit, "offset": offset,
                           "total": len(items)}}

@app.post("/v1/tasks", status_code=status.HTTP_201_CREATED)
def create_task(body: TaskIn, response: Response):
    task = Task(id=str(uuid4()), **body.model_dump())
    db[task.id] = task.model_dump()
    response.headers["Location"] = f"/v1/tasks/{task.id}"   # 201 best practice
    return task

@app.get("/v1/tasks/{task_id}")
def get_task(task_id: str):
    if task_id not in db:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "task not found")
    return db[task_id]

@app.patch("/v1/tasks/{task_id}")           # partial update
def update_task(task_id: str, body: TaskIn):
    if task_id not in db:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "task not found")
    db[task_id].update(body.model_dump())
    return db[task_id]

@app.delete("/v1/tasks/{task_id}",
            status_code=status.HTTP_204_NO_CONTENT)
def delete_task(task_id: str):
    db.pop(task_id, None)                   # 204: no body on success
```

BASH

```
# Run it, then explore the auto-generated docs at /docs
uvicorn main:app --reload
curl -X POST localhost:8000/v1/tasks -H "Content-Type: application/json" \
     -d '{"title":"Write the REST guide"}'
```

✓ **Nice**

FastAPI generates an interactive OpenAPI (Swagger) UI at `/docs` automatically from your type hints — documentation and validation for free.

Designing and Documenting with OpenAPI

OpenAPI (formerly Swagger) is the de-facto standard for describing REST APIs in a machine-readable YAML/JSON document. From one spec you can generate interactive docs, client SDKs, server stubs, and contract tests. Design-first teams write the spec before the code.

YAML

```yaml
openapi: 3.1.0
info:
  title: Tasks API
  version: 1.0.0
paths:
  /v1/tasks:
    get:
      summary: List tasks
      parameters:
        - { name: limit, in: query, schema: { type: integer, default: 20 } }
      responses:
        "200":
          description: A page of tasks
          content:
            application/json:
              schema: { $ref: "#/components/schemas/TaskList" }
    post:
      summary: Create a task
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: "#/components/schemas/TaskIn" }
      responses:
        "201": { description: Created }
components:
  schemas:
    TaskIn:
      type: object
      required: [title]
      properties:
        title: { type: string, maxLength: 120 }
        done:  { type: boolean, default: false }
```



Testing REST APIs

Test at multiple levels: unit tests for handlers, integration tests that hit the real HTTP layer, and contract tests that verify you still match the OpenAPI spec. Below, a Python example using `pytest` + FastAPI's test client.

PYTHON

```python
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_create_and_fetch_task():
    # Create → expect 201 and a Location header
    r = client.post("/v1/tasks", json={"title": "ship it"})
    assert r.status_code == 201
    task = r.json()
    assert task["title"] == "ship it"
    assert "Location" in r.headers

    # Fetch the created resource → 200
    r2 = client.get(f"/v1/tasks/{task['id']}")
    assert r2.status_code == 200
    assert r2.json()["done"] is False

def test_missing_task_returns_404():
    r = client.get("/v1/tasks/does-not-exist")
    assert r.status_code == 404

def test_validation_rejects_empty_title():
    r = client.post("/v1/tasks", json={"title": ""})
    assert r.status_code == 422   # Pydantic validation error
```

Tip

Add a smoke-test suite that runs against a deployed environment with real auth. Tools like **Postman/Newman**, **Schemathesis** (property-based from your OpenAPI spec), and **Pact** (consumer-driven contracts) catch regressions curl never will.

Best Practices and Common Pitfalls

DO

Use nouns + proper verbs, correct status codes, consistent JSON casing, pagination on all lists, HTTPS everywhere, and versioned URIs.

DO

Return a `Location` header on 201, a standard error envelope, and rate-limit headers (`X-RateLimit-*` , `Retry-After`).

✓ **Best practice**

Make GET, PUT, and DELETE idempotent and GET side-effect-free. Design for backward compatibility: add fields, never repurpose or remove them within a version.

⚠ **Pitfall**

Returning `200 OK` for errors ("tunneling" errors in the body) breaks clients, caches, and monitoring. Let the status code tell the truth.

✕ **Pitfall**

Verbs in URIs (`/createUser`), chatty N+1 round trips, leaking stack traces in 500s, unbounded list endpoints, and putting secrets in query strings (they land in logs). Avoid all five.

Interview Questions & Answers

What makes an API "RESTful"?

It follows REST's constraints: a client-server split, statelessness, cacheability, a layered system, and — above all — a uniform interface built on resources (addressed by URIs) manipulated with standard HTTP verbs that exchange self-descriptive representations. In practice this means nouns in URIs, correct verbs and status codes, and stateless requests.

Explain safe vs idempotent methods.

A safe method does not modify server state (GET, HEAD, OPTIONS). An idempotent method yields the same server state whether called once or N times (GET, PUT, DELETE, HEAD). All safe methods are idempotent, but not vice versa — PUT is idempotent yet not safe. POST is neither.

PUT vs PATCH — when do you use each?

PUT replaces the entire resource with the representation you send; omitted fields are cleared. PATCH applies a partial modification, changing only the fields you include. Use PUT when the client holds the full state, PATCH for targeted updates.

POST returned a timeout but may have succeeded. How do you retry safely?

Use an idempotency key: the client sends a unique `Idempotency-Key` header; the server stores the result against that key on first processing and replays the stored response on any retry, so no duplicate resource is created.

What is the difference between 401 and 403?

401 Unauthorized means authentication is missing or invalid — the server does not know who you are. 403 Forbidden means you are authenticated but lack permission for this action. 401 is about identity; 403 is about authorization.

How does a JWT work, and what are its risks?

A JWT is a signed token of three Base64URL parts — header, payload (claims like `sub`, `exp`, scopes), and signature. The server verifies the signature with a secret or public key, so it trusts the claims without a DB lookup (stateless). Risks: the payload is readable (not encrypted), tokens cannot be revoked individually before expiry, and a leaked signing key is catastrophic. Mitigate with short expiry, refresh tokens, HTTPS, and a rotation/denylist strategy.

Offset vs cursor pagination?

Offset/limit is simple and lets you jump to any page, but it degrades on large tables and can skip or duplicate rows when data changes mid-scroll. Cursor (keyset) pagination encodes the position of the last item, giving stable, fast, O(log n) reads at scale — at the cost of not being able to jump to an arbitrary page.

How do you version a REST API without breaking clients?

Bump a major version (commonly in the URI path, `/v2`) only for breaking changes. Within a version, evolve additively — add new optional fields and endpoints, never remove or repurpose existing ones. Communicate deprecation with a timeline and the `Deprecation` / `Sunset` headers.

REST vs GraphQL — when would you choose each?

REST shines with resource-oriented, cacheable APIs and a stable, well-understood surface; HTTP caching and tooling come for free. GraphQL lets clients request exactly the fields they need in one round trip, avoiding over/under-fetching — valuable for complex, nested data and many client types. GraphQL trades HTTP-level caching and simplicity for flexibility.

What is HATEOAS and why is it rare?

HATEOAS (Hypermedia As The Engine Of Application State) means responses include links telling the client what actions are available next, so clients navigate by following links rather than hard-coding URIs. It is the highest REST maturity level but rare because it adds complexity and most clients are built against documented, fixed endpoints instead.

How do you handle rate limiting?

Return `429 Too Many Requests` when a client exceeds its quota, include `Retry-After` telling it when to try again, and expose `X-RateLimit-Limit/Remaining/Reset` headers so well-behaved clients can self-throttle. Common algorithms are token bucket and sliding window.

> **Why must REST be stateless, and where does session state live?**
>
> Statelessness means each request carries all context (e.g. an auth token), so any server instance can handle any request — enabling horizontal scaling, simple load balancing, and resilience. State that must persist (user sessions, carts) lives in the client (tokens) or a shared store (database, Redis), not in per-server memory.

Further Learning

- Read Roy Fielding's dissertation, chapter 5, for REST's original rationale.
- Study the HTTP semantics specs: RFC 9110 (HTTP semantics) and RFC 9457 (Problem Details for error bodies).
- Build the Tasks API above, then add JWT auth, cursor pagination, and an OpenAPI spec; generate a client SDK from it.
- Explore adjacent styles — GraphQL, gRPC, and webhooks — to understand REST's trade-offs.
- Practice with Schemathesis and Pact to make contract testing part of your CI pipeline.
- Review real-world API design guides from Stripe, GitHub, and Google (AIPs) — they are masterclasses in pragmatic REST.